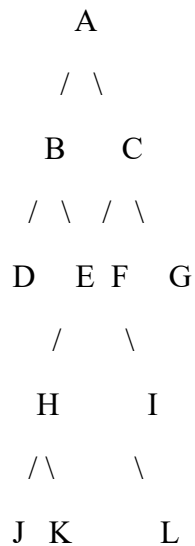


Work Sheet 6 (Data Structures)

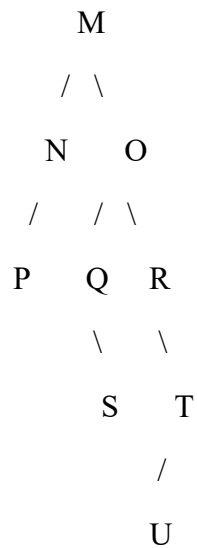
Part A:

1. **Tree Identification:** For the given trees, determine whether they are **full**, **perfect**, **complete**, or **balanced**. Justify your answers.

- **Tree 1:**



- **Tree 2:**



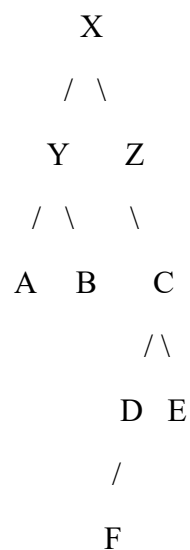
2. **Tree Properties:** For **Tree 1**, determine:

- The height of the tree.
- The total number of nodes.
- The number of leaf nodes.
- The depth of node H.
- The degree of node B.

For **Tree 2**, determine:

- The depth of node U.
 - The ancestors of node T.
 - The siblings of node R.
 - Whether the tree is balanced or not (explain why).
-

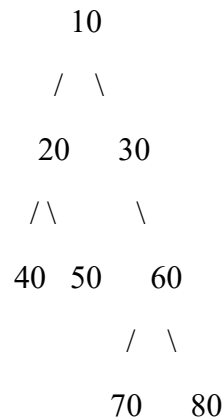
3. **Subtree Identification:** Given the following tree:



- Identify the subtree rooted at Z.
- List all descendants of node Y.
- Determine whether the subtree rooted at C is a **perfect binary tree**.

4. Traversal Analysis:

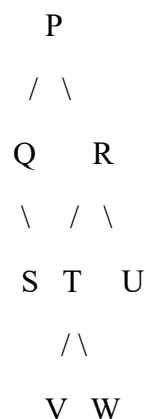
- Given the tree below:



- Manually compute the **preorder**, **inorder**, and **postorder** traversals. Explain how the order of traversal differs for each method.
- Which traversal would be most suitable for reconstructing the tree if only traversal output is provided? Why?

5. Deletion Scenarios: Analyze and solve the following deletion cases for the given trees:

- **Scenario 1 (Leaf Node Deletion):** In **Tree 1**, delete the node L. What steps are taken, and how does the tree structure change after deletion? Explain if this impacts the tree being complete or balanced.
- **Scenario 2 (Node with One Child):** In **Tree 2**, delete the node T. How do you handle the subtree rooted at U, and how does the structure of the tree change? Justify whether the tree remains balanced after this operation.
- **Scenario 3 (Node with Two Children):** For the following tree, delete the node R:

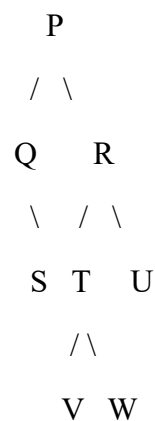


Steps to consider:

- Identify the appropriate replacement node for R.
 - Explain the process of reattaching subtrees after deletion.
 - Determine whether this operation introduces imbalance to the tree. If so, how can it be rectified?
-

6. Conceptual Questions:

- In **Tree 1**, if the height of the left subtree of B is reduced by 1, will the tree remain balanced? Justify your answer.
- In **Tree 2**, if a new node V is added as the left child of U, will the tree become a full binary tree? Why or why not?
- In **Tree 1**, is it possible to convert the tree into a perfect binary tree by adding exactly two nodes? If yes, where should these nodes be added?
- For the tree below:



Determine:

- The maximum height difference between the left and right subtrees for all nodes. Is the tree balanced based on this analysis?
- How many nodes need to be added to make the tree complete?

Part B:

Binary Search Tree ADT

Implement an Abstract Data Type (ADT) for a Binary Search Tree (BST) using nodes that store data as a separate class object.

Data Class:

Define a Record class with the following attributes:

- `record_id` (int): A unique identifier for the record.
- `name` (str): Name of the record holder.
- `age` (int): Age of the record holder.
- `address` (str): Address of the record holder.

Include a method to initialize the Record object and print its details for debugging or display purposes.

Node Class:

Define a Node class for the tree node. Each node should include:

- `data` (Record): An object of type Record representing the record stored in the node.
 - `left` (Node): Pointer to the left child node (initially None).
 - `right` (Node): Pointer to the right child node (initially None).
-

BinarySearchTree Class:

Define a BinarySearchTree class with:

- `root` (Node): Pointer to the root node (initially None).

Include the following core methods:

1. **Insert:** Add a record to the tree using `record_id` for ordering.
2. **Find:** Search for a record by its `record_id`.
3. **Delete:** Remove a record identified by `record_id`. Handle all cases (leaf node, one child, two children).
4. **Traverse and Display:** Print all records in ascending order of `record_id` using **Inorder Traversal**.

Task:

1. **Data Insertion:** Insert the following records into the BST:

(101, "Alice", 25, "123 Main St"),
(205, "Bob", 30, "456 Elm St"),
(150, "Charlie", 22, "789 Pine St"),
(50, "Diana", 28, "321 Oak St"),
(75, "Eve", 35, "654 Maple St").

2. **Operations:**

- Search for the record with record_id = 150 and display all its details.
- Delete the record with record_id = 205. Display the tree using **Inorder Traversal** after deletion.
- Insert a new record: (180, "Frank", 29, "987 Willow St"). Display the updated tree.
- Determine the **height** of the BST after these operations.

Analysis Questions:

- How does the tree structure change after the deletion of a node with two children?
 - Is the tree balanced after these operations? Justify your answer.
-

Part C:

Advanced Thinking and implementation:

- Consider part B, If the database user wants to search for records not only by record_id but also by **age**, discuss:
 - Why the current BST structure might not be efficient for searching by multiple attributes?
 - Suggest possible approaches or data structures that could handle searches by multiple attributes (e.g., record_id, age, or name) efficiently.
 - Implement a function to check whether a given binary tree is a valid BST. Test your function with multiple examples.
-

Part D:

Conclude the difference of approach for deletion:

- Consider following visualizers, create a tree of your choice and conclude the deletion process they are using for deleting nodes:
 - <https://visualgo.net/en/bst>
 - <https://www.cs.usfca.edu/~galles/visualization/BST.html>